Multi-Process Command Pipeline System

Presented by

Svara Hassini
2520090170

Sudharshini
2520090227

Sumanjali
2520090190

Problem Statement

Running a chain of commands needs more than one process working together.

We build a system that connects the output of one command to the input of the next, like a shell pipeline.

1

Single process is slow

Commands run one after another, so the work cannot overlap.

2

Data must flow between commands

Output of one command has to reach the next without temp files.

3

Process management is hard

Creating, linking and cleaning up many processes needs care.

System Design



Processes 1 to 3 are connected by pipes: stdout of one becomes stdin of the next.

Parent process

Reads the command line, creates pipes, forks children and waits for them.

Child processes

Each child redirects its input and output, then runs its command.

Pipes

One-way channels that carry data between neighbouring processes.

Implementation Approach

1. **Parse input**
   Split the line at each | into separate commands and arguments.

2. **Create pipes**
   Make N-1 pipes for N commands using pipe().

3. **Fork processes**
   Use fork() to create one child per command.

4. **Redirect I/O**
   Connect pipe ends to stdin and stdout with dup2().

5. **Execute**
   Each child runs its command with exec(), then the parent closes unused pipe ends.

6. **Wait and clean up**
   Parent calls wait() on every child and reports exit status.

Technologies Used

>_

C / C++

Core language for the pipeline program

>_

POSIX system calls

fork, pipe, dup2, exec, wait

>_

Linux / Unix

Operating system environment

>_

GCC and Make

Compile and build the project

>_

GDB and Valgrind

Debugging and leak checking

>_

Git

Version control for the team

Testing

Test case	Example	What we checked
Single command	`ls`	Runs normally with no pipe
Two-stage pipe	`ls | wc -l`	Output passes between 2 processes
Multi-stage pipe	`cat f | sort | uniq`	Data flows through 3 processes
Invalid command	`abc | ls`	Error shown, no crash or hang
Large output	`yes | head -n 100000`	Pipe buffer handled correctly
Cleanup	`any pipeline`	No zombie processes or open pipe ends

Results

3+

commands chained in one pipeline

0

zombie processes after completion

100%

of test cases behaved as expected

- Commands run in parallel, so each stage starts working as soon as data arrives.
- Data moves through pipes in memory, with no temporary files.
- Errors in one command are reported without crashing the whole system.

Impact

1

Better performance

Parallel processes overlap their work and use the CPU more efficiently.

2

Clear OS concepts

Shows how processes, pipes and inter-process communication really work.

3

Reusable design

The same model can be extended with redirection, background jobs and more.

4

Foundation for shells

A base for building a full command-line shell.

Conclusion

- Built a multi-process pipeline using fork, pipe, dup2 and exec.
- Tested with single, multi-stage and error cases.
- Future work: I/O redirection, background jobs and built-in commands.

Thank you

Svara Hassini | Sudharshini | Sumanjali